

Reflection: Evaluating the $P + x$ Transformation

Adjusting the offset value x in a neural language model drastically shifts the syntactic coherence and semantic tone of Wallace Stevens' *The Snow Man*. At a low rank such as $P + 7$, the predicted tokens remain within the high-probability topk distribution. Because causal models like GPT-2 assign high likelihood to grammatically natural continuations, $P + 7$ retains strong thematic alignment with the original context. The replaced words often function smoothly as partial synonyms, syntactically compatible nouns, or plausible poetic variations (e.g., yielding words like "*her*" or "*trees*"), creating an output that reads like a parallel draft of the poem.

As x increases to higher offsets like $P + 15$ or $P + 23$, the model is forced to draw from the long tail of its probability distribution. At these ranks, predictions shift away from likely contextual continuations toward lower-probability function words, subword fragments (e.g., *gro*, *ut*), and semantically distant terms. The syntactic structure degrades rapidly: lines often end in prepositions or truncated prefixes rather than meaningful substantive words. While this increasing entropy breaks the poem's grammatical illusion, it illuminates the inner mechanics of Byte Pair Encoding (BPE) tokenization and autoregressive logit scoring, showing how neural models shift from fluent generation to random subword sampling as rank depth increases.

Implementing a Strict Noun-Replacement $P + 7$ Technique

To implement a true Oulipian $P + 7$ constraint where **only nouns** are replaced by their 7th-highest-probability **noun** candidates, the script requires a part-of-speech (POS) filtering pipeline:

1. **POS Tagging:** Parse each line using an NLP library (such as `nltk.pos_tag` or `spaCy`) to identify the specific POS tag of the final word. If the target word is not a noun (NN, NNS, NNP), it remains untouched.
2. **Vocabulary Filtering:** Collect GPT-2's output logits for the target position, apply `torch.softmax` to derive probabilities, and iterate through the sorted candidate list using `torch.topk`.
3. **Noun Selection:** Filter candidate tokens to ensure they are complete alphabetic words (`token_str.startswith('Ġ')`) and verify using `nltk.pos_tag` that the candidate is also a noun.
4. **Rank Extraction:** Select the 7th candidate that satisfies the noun criteria, preserving the original line's trailing punctuation and grammatical structure.